The Illusion of Passing Tests[1]

Jason Tang
CBAI AIxBio

With
Apart Research

Abstract

In secure program synthesis (SPS), the core challenge is shifting from generating code to verifying what AI actually writes before it is authorized for deployment. As large language models improve, they increasingly expose a gap between passing a visible test and satisfying the behavior a security-sensitive component is meant to enforce. This creates the illusion of passing tests: the false belief that visible test success is enough to justify authorizing code to cross a security-relevant trust boundary. Advancing toward rigorous vericoding [10] therefore requires re-evaluating what counts as sufficient evidence to release or deploy model-written code. Because complete formal specifications are difficult to elicit for arbitrary systems [4], we cannot rely exclusively on end-to-end proofs. We study a concrete trust-boundary triage workflow that treats specifications as executable oracles [14]. We evaluate this deployment-gate workflow on a frozen suite of 24 narrow-waist, security-relevant Python components at security-sensitive boundaries. Across our confirmatory set, candidates on all 16 tasks passed visible tests but failed hidden checks, including 8 instances of security-critical failures. Specification-derived executable oracles proved most valuable not by improving scores, but by blocking insecure code. A tests-only selector falsely authorized insecure artifacts on 7 tasks, whereas our specification-aware triage layer reduced this to a single false accept. However, consistent with Trusted Computing Base (TCB) literature [6, 8], stronger automated checks still do not close the deployment problem: our system explicitly escalated 6 artifacts to human review, making clear that passing automated checks does not resolve residual interface and trust-boundary assumptions. Our findings argue that SPS needs a dedicated deployment gate for security-relevant code, where specifications matter operationally by rejecting false sufficiency, blocking insecure code, and surfacing the boundaries where human review is still required.

[1]Research conducted at the SPS Hackathon, May 2026. Repo: `github.com/davidkimai/sps_evals`.

1. Introduction

Passing tests does not guarantee deployment readiness. The core problem in autonomous software engineering is the illusion of passing tests: the false belief that visible test success is sufficient evidence to authorize AI-generated code to cross a security-relevant trust boundary.

As frontier models mature, it is no longer difficult to generate plausible, compiling code that satisfies a visible test harness. But Secure Program Synthesis (SPS) is not only about generation; it is about deciding what can be safely released or deployed. Recent work in the SPS community highlights that the bottleneck has shifted from eliciting code to specifying and validating its behavior [3, 1]. When a coding agent can rapidly generate plausible code at scale, the critical question becomes: what evidence justifies letting that artifact past a deployment gate?

This false confidence persists because we mistake a proxy for the target. While unit tests are standard proxies for correctness, they are often model-facing, narrow, and easily satisfied without capturing the full intended behavior. Regehr's "executable oracle" framework illustrates this vulnerability: LLM outputs can pose risks when they retain sufficient degrees of freedom to satisfy a weak proxy while introducing subtle flaws [14]. A few passing tests might prove the code handles the cases we explicitly remembered; they do not prove the code is structurally sound or secure.

Furthermore, even formal, strong-looking evidence can induce false confidence. As Mike Dodds points out, coherent, global specifications simply do not exist for arbitrary, real-world systems [4]. Expecting an LLM to formally verify an entire monolithic application is an unrealistic assumption. Instead, verification claims are only as meaningful as their Trusted Computing Base (TCB), interfaces, and human review assumptions allow them to be [6]. Recent security audits of "high-assurance" cryptography underscore this operational reality: carrying a proof or passing a rigorous formal check does not automatically render an artifact safe to deploy [8]. The challenge, then, is achieving justified deployment authorization under bounded evidence.

This paper tackles that challenge on a concrete, security-sensitive slice. Rather than attempting to solve SPS in full generality, we restrict our focus to a frozen suite of narrow-waist Python components that behave like small defensive primitives: authorization logic, rate limiting, parser boundaries, and resource scoping. In this bounded setting, the code sits on trust boundaries, failures have immediate security consequences, and executable checks remain meaningful. For each task, we generate multiple candidate implementations and subject them to ordinary visible tests. Crucially, we then apply stronger, hidden executable checks derived from the underlying specification. Finally, we force the system into a strict trust-boundary triage: `auto_accept`, `auto_reject`, or `escalate_to_review`. The objective is not to publish another code generation benchmark score; it is to study a deployment gate for AI-generated security-relevant code.

This framing deliberately distances our work from both traditional code-generation benchmarks and unbounded formal-synthesis ambition. Unlike standard benchmarks, we care less about whether a model *can* emit a passing artifact and more about whether a tests-only release signal is actually safe to trust. And unlike end-to-end vericoding efforts [10], we deploy specifications as localized evaluators over a pool of candidates rather than as complete formal objects for whole-program synthesis. We are asking how to operationally verify what an AI writes today before it crosses a trust boundary, rather than how to extract slightly better code from a prompt.

Our central empirical finding confirms the SPS hypothesis: visible tests routinely create a false impression of sufficiency. Across our confirmatory dataset, we observe pervasive visible-pass / hidden-fail behavior—including severe security failures that fully satisfied the visible harness. In this triage workflow, hidden evaluators prove most valuable for breaking this false confidence and saying "no." By filtering out artifacts that a tests-only baseline would blindly accept, specification-aware triage dramatically

reduces dangerous false accepts.

However, our results also establish a firm boundary on what stronger evaluation can accomplish. A selector can only rank what has been generated. Where candidate generation fails to surface a genuinely correct solution, advanced evaluation cannot manufacture correctness. Finally, our findings reinforce the necessity of human oversight: stronger automated checks do not eliminate review obligations. We demonstrate that some selected artifacts must still be explicitly escalated to human review, as passing an executable oracle does not resolve all trust-boundary and wrapper assumptions.

Our contribution is not a claim that we have solved secure program synthesis. Rather, we establish that deployment authorization must be treated as a first-class systems problem for security-relevant code. In this paradigm, specifications are not decorative prompt texts; they are executable evaluators that catch hidden failures, block dangerous false accepts, and cleanly identify the operational boundaries where human judgement is still required.

Our main contributions are as follows:

1. **A trust-boundary deployment-gate framing for Secure Program Synthesis.** We formalize a triage workflow for AI-generated code that outputs an operational decision (`auto_accept`, `auto_reject`, or `escalate_to_review`) rather than a mere benchmark score.

2. **A frozen, narrow-waist evaluation suite.** We anchor our study on a 24-task primary suite of security-relevant Python components, complemented by a 24-task expansion suite to stress-test triage mechanisms and review boundaries.

3. **Evidence that visible tests are weak deployment proxies at trust boundaries.** Across the confirmatory set, we document pervasive visible-pass / hidden-fail behavior, including multiple instances where visibly passing code harbored hidden security vulnerabilities.

4. **A strong secure-rejection result with a clear mechanism boundary.** Hidden specification-based evaluators drastically reduce secure false accepts. Crucially, our analysis confirms that ranking mechanisms are only effective when candidate generation provides hidden-correct support.

5. **An explicit review-boundary result.** We show that passing automated checks is insufficient for complete trust. Several selected artifacts required explicit human review, underscoring the SPS principle that strong evidence does not negate the need for human verification.

2. Related Work

The Specification Bottleneck in Secure Program Synthesis. As the SPS community increasingly recognizes, the primary challenge of AI-assisted coding is shifting from generation to validation. Recent literature asserts that determining what constitutes trustworthy evidence is now the critical bottleneck [3, 1]. Compounding this is Dodds' observation that complete, formal specifications rarely exist in the wild for arbitrary software systems [4]. This reality justifies our narrow-waist scope: we isolate components where the intended behavior is sharp enough to explicitly test and review. Furthermore, tools like Atlas's *formal-specification-ide* validate the premise that specifications must be treated as first-class, inspectable objects rather than after-the-fact inferences [5]. Our trust-boundary triage workflow directly operationalizes this worldview.

Vericoding and Specification-Guided Search. The recent introduction of the Vericoding benchmark [10] establishes formally verified program synthesis as a primary evaluation vector, distinguishing rigorous synthesis from informal generation. This modern push inherits from a long lineage of specification-guided methods, including Counterexample-Guided Abstraction Refinement (CEGAR) [11]

and protocol synthesis [12], which demonstrated the power of explicit counterexamples. Our work is philosophically aligned with this tradition but addresses a more targeted operational question. Rather than focusing on end-to-end formal synthesis, we investigate how specification-based evaluators perform as a strict triage layer over an existing pool of candidate programs.

Executable Oracles and Model Constraint. Regehr's "executable oracle" framework [14] directly informs our methodology. Regehr posits that autonomous coding agents introduce significant risk when they retain too many degrees of freedom to fulfill weak tests while hiding structural flaws. Our empirical results serve as a deployment-gate validation of this argument. While visible tests technically function as oracles, they are often weak proxies. Stronger, hidden executable checks constrain the agent's generation, serving not just to rank better candidates, but critically, to reject deceptive ones that would otherwise cross the trust boundary.

Deployment-Gating vs. Generation-Time Alignment. Research into methods like Approximately Aligned Decoding [13] explores how to structurally enforce constraints during the model's generation phase. While crucial, our research operates at a distinctly different intervention point: the deployment pipeline. By utilizing fixed candidate pools and treating triage as a distinct, post-generation decision gate, we decouple the quality of the raw generation from the rigor of the deployment criteria. This isolates the specific role of evidentiary boundaries in deployment decisions.

The Review Boundary and False Assurance. Finally, our `escalate_to_review` metric is heavily informed by critiques of high-assurance claims and Trusted Computing Bases (TCBs). Tools such as *lean-tcb* actively interrogate what assumptions a human must still manually review for a formal proof to be practically meaningful [6]. Corresponding critiques of "high-assurance" cryptography emphasize that verified claims often obscure unreviewed wrapper vulnerabilities and interface assumptions [8]. Our triage system internalizes this lesson: passing rigorous automated checks is insufficient. A responsible deployment-gate system must proactively expose, rather than obfuscate, the boundaries where human review remains essential.

3. Methods: Designing an Executable Trust-Boundary Gate

Our methodology separates generation from validation. Rather than measuring an LLM's raw generation capacity, we evaluate whether a structured deployment gate can reliably triage candidate implementations. The objective is to determine how treating specifications as executable oracles affects the decision to accept, reject, or escalate an AI-generated artifact.

The Narrow-Waist Task Suite. We anchor our evaluation on a primary suite of 24 narrow-waist, security-critical Python components (comprising 8 development and 16 confirmatory tasks). These tasks govern authorization logic, rate limiting, parser boundaries, and resource scoping. By design, these components feature strict trust boundaries where failures carry security implications. We explicitly avoid full-stack applications to prevent the "coherent specification" trap [4]. A supplementary 24-task expansion suite provides additional stress-testing, though our primary claims rest on the confirmatory set.

The specification surfaces themselves are versioned in the codebase. For the owned internal slice, the bounded specs live in repo-authored task files under `data/slopbench/`, each of which packages a visible prompt, visible executable harness, and frozen day-2 harness (for example, `030_permission_gate.yaml`). For the secure slice, the public specification summaries live in a versioned JSON spec file, while the stronger hidden evaluators are generated deterministically from frozen templates in a secure-template module and a hidden-oracle module.

Candidate Pool Generation. To evaluate the triage layer against realistic outputs, we construct a diverse, bounded pool of candidate implementations for each task. Rather than relying on a single zero-shot output, we utilize an array of prompts (e.g., requirements-first, invariants-first, self-critique, and

regression-preservation templates). Our goal is not unbounded search, but to create a structurally varied candidate pool that represents the realistic output of a modern, agentic coding workflow.

Visible Tests vs. Executable Oracles. The critical intervention in our workflow is the dual-stage evaluation. First, candidates confront the **visible checks**: the standard suite of unit tests and regression signals that a typical model-facing benchmark would expose. Next, the artifacts face **hidden executable checks**. These are strictly derived from the underlying specification and deliberately concealed from the generation process. Following Regehr [14], they act as strict constraints, enforcing semantic rules and security invariants that standard harnesses may overlook.

The hidden layer is not an ad hoc bag of extra tests. On the owned internal slice, it is the frozen day-2 executable harness attached to each narrow-waist task. On the secure slice, it is a deterministic hidden oracle generated from frozen task templates and executed in an isolated workspace, with oracle hashes recorded in the ledgers. Appendix A describes this construction and validation process in more detail.

This dichotomy forms the empirical backbone of the paper, designed to explicitly test this gap by identifying exactly which artifacts satisfy the proxy but fail the target.

Table 1: Representative visible vs. hidden evidence surfaces. Full construction details appear in Appendix A.

Task surface	Visible evidence	Hidden evidence	Why hidden evidence matters
`permission_gate` (owned internal)	Explicit-role deny-by-default behavior	Frozen day-2 harness adds wildcard-role expansion while preserving deny-by-default semantics	Catches day-2 authorization behavior that a visibly correct implementation can still miss
`safe_path_validation` (secure slice)	Public summary plus schema-level visible test	Deterministic hidden oracle checks that adversarial paths such as `"../secret"` are rejected	Tests whether the component actually enforces the security boundary rather than only returning the right schema

Selection on Fixed Pools. To isolate the efficacy of our deployment gate, we evaluate competing selectors against the identical, fixed pools of generated candidates. This guarantees that any performance gains stem strictly from superior specification-aware evaluation, not from stochastic variances in generation. Furthermore, all selector inputs are anonymized to prevent bias toward specific prompt lineages.

Trust-Boundary Triage Decisions. Instead of emitting a continuous score or a binary pass/fail, our deployment gate strictly triages each selected artifact into one of three operational decisions:

- **`auto_accept`:** The artifact successfully passes all visible, hidden, security, and regression checks. Within the bounded task setting, the specification is satisfied and no residual trust-boundary concerns are surfaced by the gate.
- **`auto_reject`:** The artifact fails any check layer (parse, runtime, visible, hidden, or security). The executable oracle blocks the artifact from crossing the deployment gate.
- **`escalate_to_review`:** The artifact passes all automated oracles, but surfaces a structural ambiguity regarding interface assumptions, wrappers, or the Trusted Computing Base (TCB). A human must adjudicate the remaining uncertainty [6].

Bounded Repair and Manual Adjudication. We also conduct a bounded repair comparison, evaluating whether investing compute in localized repair loops can recover hard failures better than fresh generation. Finally, to ensure scientific integrity, we manually adjudicate the flagship visible-pass / hidden-fail instances, false accepts, and review escalations. This guarantees our claims are rooted in genuine semantic and security failures, rather than mere test-harness flakiness.

Provenance and Reproducibility. Because deployment authorization is the central thesis of our work, all triage decisions are logged in append-only ledgers. To ensure our evaluation substrate aligns with modern frontier-eval standards, the claim-bearing workflow was executed and logged using Inspect AI [15], providing rigorous runtime provenance and reviewability. Table 2 details the scale of our packaged dataset.

Table 2: Evidence scale for the packaged v3 study.

Evidence surface	Count
Total tasks	48
Main evaluation tasks	24
Expansion tasks	24
Candidate rows	1405
Selector rows	360
End-to-end rows	360
Secure-eval rows	369
Triage decisions	72
Deep manual adjudication rows	138

4. Results

Our empirical results demonstrate a clear operational requirement: deployment authorization for security-relevant code needs stronger evidence than visible tests alone. Unit tests identify superficial errors, but hidden executable oracles act as security checkpoints at the trust boundary and reject insecure code that a visible harness would otherwise wave through. The same results also establish the evaluator's boundary: triage only works when generation has already placed an evaluator-passing implementation into the candidate bank, and formal validation still leaves a human review frontier.

Act 1. The Illusion in Practice: Visible-Pass / Hidden-Fail

A key finding of this study is the prevalence of this proxy-target gap. Across all 16 confirmatory tasks, the candidate pools produced artifacts that successfully passed the visible test harness but failed the stricter, hidden executable checks. Specifically, on 8 of those tasks, the hidden failures involved security invariants.

This highlights the core SPS deployment-gate challenge: code may satisfy standard test harnesses while remaining structurally unsafe. Relying solely on model-facing tests conflates proxy success with target satisfaction. In a zero-degree-of-freedom paradigm [14], visible tests simply leave the agent with too much maneuverability.

Act 2. Specifications Earn Their Keep by Saying "No"

The primary impact of specification-derived oracles was the suppression of false accepts. Relying exclusively on tests-only selection, our system authorized secure failures on 7 distinct tasks. When we

enforced the specification-aware evaluation, that failure rate dropped to 1.

Table 3: Secure false accepts under two selection strategies.

Measure	Count
Secure false-accept tasks under tests-only selection	7
Secure false-accept tasks under spec-aware selection	1
Tasks on which secure false accepts were reduced	6

This reduction carries direct operational relevance. The primary utility of a hidden executable oracle is overriding visible test success to reject an artifact that poses a security risk. In the context of autonomous agents, the specification's highest value is its capacity to say "no."

Act 3. The Support Bottleneck: Oracles Cannot Manufacture Code

Our results also delineate the functional limits of evaluation layers. Analyzing the support cases, we found 8 tasks where the candidate pool successfully generated a hidden-correct solution, and 9 where it failed entirely. Upgrading to a specification-aware selector beat the baseline on only 2 tasks. In a subsequent deep-dive across 8 internally rigorous tasks, additional search compute failed to yield a single newly supported task.

The support-absent bucket was not random. It concentrated in stateful or interface-sensitive components where the hidden requirement depended on precedence rules, retained state, or trust-boundary-preserving generalization rather than a single visible success case. In those settings, the problem was not choosing among many plausible secure candidates; it was failing to generate one at all.

The implication is clear: specification-based triage depends fundamentally on candidate generation. A selector can only surface a secure artifact if one exists in the pool. When structural support is absent, the bottleneck is generation capability, not evaluation accuracy.

Act 4. The TCB Reality: Escalating to Human Review

Across 72 distinct accept/reject/review decisions, our system explicitly escalated 6 selected artifacts to human review. Crucially, these were not artifacts that failed tests; they were artifacts that *passed* all automated oracles but retained unresolved trust-boundary ambiguities. This demonstrates that even surviving rigorous specification checks does not erase all review obligations. These included unverified wrapper dependencies, interface assumptions, and under-specified edge cases.

This aligns with the principles of *lean-tcb* [6]: successful automated checks do not eliminate the Trusted Computing Base. A realistic, high-assurance SPS workflow must embrace a tertiary outcome. Sometimes the correct answer is not "pass" or "fail," but rather: "The automated evidence is mathematically sound, but a human must audit the interface before deployment."

A bounded formal overlay points to the same lesson. In the retained Dafny baseline, 52 of 150 candidate artifacts achieved verified status, yet all 52 still received the lowest task-level judge score in our audit because the dominant failures appeared at the operational Python boundary rather than inside the locally verified property. Verification did not erase the deployment problem; it relocated it to wrappers, interfaces, and other review-boundary surfaces.

Act 5. Bounded Repair Did Not Rescue Missing Support

Finally, we investigated whether bounded repair loops could synthesize support where zero-shot generation failed. Under a strict equal-compute budget comparison, repair won 0 times against fresh generation.

This finding suggests that lightweight, iterative repair loops cannot automatically salvage a task that lacks a fundamentally correct baseline. Both generation and bounded repair ultimately crashed into the same underlying limitation: the LLM's underlying inability to author structurally sound code on the hardest tasks.

This is consistent with how the repair arm is defined in our pipeline. Under the frozen equal-cost protocol, repair receives one bounded opportunity to patch the selected artifact using failure context rather than explore a new structural hypothesis. That can fix local mistakes, but many hidden failures in this study were not local mistakes: they reflected wrong rule ordering, weak canonicalization, missing boundary checks, or interface assumptions that required a different candidate family rather than a small edit. We therefore interpret the null result narrowly. It is not evidence against repair in general; it is evidence that cheap, local repair is not a free substitute for candidate support on hard trust-boundary tasks.

Table 4: Main findings at a glance.

Finding	Result
Visible-pass / hidden-fail behavior	Observed on all 16 confirmatory tasks
Visible-pass / hidden-secure-fail behavior	Observed on 8 confirmatory tasks
Secure false accepts under tests-only selection	7 tasks
Secure false accepts under spec-aware selection	1 task
Review escalations	6 selected artifacts
Repair wins over equal-cost fresh generation	0

5. Discussion

The central argument of this paper is that advancing Secure Program Synthesis for security-relevant software requires building a rigorous deployment gate alongside scalable code generation. As large language models increasingly generate code that compiles, executes, and passes visible tests, generation must be paired with stronger verification and review disciplines [10]. The critical question is no longer whether an AI can write code, but what evidentiary standard justifies allowing that code to cross a trust boundary into production. Our results demonstrate that treating visible test success as sufficient evidence carries significant risk.

This highlights the operational role of hidden executable oracles. Their primary utility is not metric optimization, but the direct enforcement of security logic. By exposing failures that standard harnesses miss, specification-derived evaluators override false confidence and block insecure code from deployment. A key outcome of this study is the secure-rejection result: an effective oracle is most valuable when rejecting code that appears successful but is structurally flawed.

However, our findings also highlight the functional limits of evaluation. Triage is not generative. A specification-aware selector cannot manifest a secure, correct artifact if the generator failed to produce one. The lack of candidate support on high-complexity tasks confirms that better evaluation cannot fully compensate for inadequate generation. If the candidate pool lacks a valid solution, no oracle can save the system.

Furthermore, our explicit escalation metric highlights a structural reality: strong automated evaluation does not negate the need for manual review boundaries. When our system escalated artifacts that had cleared all executable oracles, it functioned as a necessary structural pause. Recognizing that "passed the mathematical checks" and "safe to ship" are distinct operational states is the bedrock of trusted computing base (TCB) theory [6, 8]. A mature deployment-gate system must structurally accommodate unresolved trust-boundary assumptions.

This review-boundary limitation is also visible in our bounded Dafny overlay. Out of 150 candidate artifacts evaluated against Dafny [9] specifications, 52 achieved a verified status. Yet all 52 still received the lowest task-level judge score in our audit, with the dominant failure mode appearing at the operational Python boundary—wrapper incompatibilities, interface mismatches, or incorrect runtime assumptions. This is not a failure of formal methods, but a practical demonstration of the review-boundary problem: mathematically sound evidence does not automatically satisfy the operational requirements for safe deployment.

A newer bounded Lean 4 formal evaluator [7] sharpens this point rather than reversing it. Concretely, this was a small mechanized formal test harness over 6 narrow-waist tasks. Visible-pass / formal-fail behavior still appeared, and once the candidate pool contained hidden-correct support the formal selector reduced bounded-slice false-accept tasks from 3 to 0. In a targeted 2-task follow-up, we then produced a confirmatory formal-pass / review-required case on `timing_safe_compare`: a candidate satisfied the visible harness and bounded formal I/O checks, yet was still escalated because the mechanized slice did not certify constant-time behavior without `compare_digest`. We read this as the most honest positive formal result in the package: narrow formal evaluators can harden the deployment gate, but their strongest operational value is still to expose residual review obligations rather than dissolve them.

Table 5 summarizes these three formal evidence surfaces. The point is not to compare them as if they were one shared benchmark denominator. The point is to show their distinct roles in the trust-boundary story: the Dafny overlay as a warning about verification-façades, the bounded Lean 4 formal-evaluator slice as a positive hardening result, and the targeted Lean 4 follow-up as a concrete review-boundary result.

Table 5: Bounded formal evidence surfaces and what they mean. These rows are grouped by evidentiary role in the acceptance story, not ranked as coequal benchmark results.

Surface	Evidentiary role	Representative signal and lesson
Historical Dafny overlay	Warning	52/150 artifacts reached verified status, but all 52 still received the lowest task-level judge score at the Python boundary; proof-bearing status can fail at wrappers, interfaces, or runtime assumptions.
Lean 4 formal evaluator (6 tasks)	Hardening result	Visible-pass / formal-fail appeared on 5 tasks, and once support existed formal selection reduced bounded-slice false-accept tasks from 3 to 0; narrow formal evaluators can strengthen an acceptance gate.
Lean 4 review-boundary follow-up (2 tasks)	Review-boundary exposure	A confirmatory formal-pass / review-required timing-safe case showed that mechanized formal evidence can still lead to `escalate_to_review` when the trust boundary extends beyond the formal harness.

This operational reality also dictates our narrow-waist methodology. Dodds correctly observes that complete, formal specifications rarely exist for sprawling, arbitrary systems [4]. Given that reality, the

pragmatic approach is not to attempt zero-shot verification of entire monoliths. Instead, progress in secure program synthesis requires isolating components with sharp, inspectable boundaries where executable checks possess genuine enforcement power. In security terms, these are small trust-boundary or choke-point components where a stronger gate can retire meaningful classes of false confidence. Our bounded scope is a deliberate architectural choice prioritizing inspectable boundaries over theoretical completeness.

Ultimately, the operational requirements are bounded. Visible tests provide weak assurance, while strong executable oracles are necessary to reject deceptive artifacts before they cross a trust boundary. Triage mechanisms only succeed when fed competent candidate pools. And even the most rigorous automated pipelines require explicit off-ramps for human review. This represents a bounded but highly operational claim. Trustworthy autonomous agents will require deployment gates structurally designed to differentiate between safe release, necessary rejection, and human escalation.

5.1 Limitations

The primary limitation of this study is its intentional, narrow-waist design. By focusing strictly on security-critical Python components with sharp trust boundaries, we trade broad applicability for deep operational clarity. We do not claim that this exact triage workflow seamlessly scales to monolithic applications, heavily stateful systems, or codebases lacking explicit, localized invariants.

Second, our evaluation layer does not resolve the generation bottleneck. On our most demanding tasks, the LLM systematically failed to generate a single hidden-correct artifact. In such support vacuums, advanced selection algorithms are rendered impotent. This paper demonstrates that specification-aware triage is necessary, but it explicitly disclaims that it is *sufficient* without concurrent advancements in generator capabilities.

Third, our implementation of bounded repair failed to act as a rescue mechanism. Under a strict compute-parity constraint, iterative repair was outmatched by fresh generation. While this does not categorically invalidate agentic repair loops, it proves that basic self-correction is not a free pass for overcoming fundamental zero-shot generation deficits in hard security tasks.

Finally, this paper does not present an end-to-end, theorem-prover-native formal synthesis breakthrough. We do not produce mathematical proofs of whole-system security. Instead, we offer an operational deployment-gate workflow for bounded tasks. Our formal-methods-adjacent conclusions focus strictly on practical review boundaries, TCB limitations, and mitigating false assurance, rather than pursuing unbounded theoretical verification.

5.2 Future Work

The most urgent vector for future research is addressing the generation bottleneck on high-complexity tasks. Because triage cannot salvage an empty candidate pool, the SPS community must develop fundamentally better structural elicitation methods that provide automated evaluators with viable candidates to discriminate among.

Second, the field requires sophisticated review-boundary tooling. Several artifacts successfully passed all automated checks but were rightfully escalated due to unresolved wrapper and TCB ambiguities. Developing explicit interfaces for surfacing, bounding, and mitigating these residual assumptions is critical for integrating formal validation into operational workflows.

Third, bounded formal evaluators now look promising on minute, narrow-waist tasks, but only as secondary surfaces. In our retained Lean 4 formal-evaluator experiments, a 6-task slice reproduced the support-conditioned ranking and false-accept suppression story under stronger mechanized checks,

while a targeted 2-task follow-up yielded a real formal-pass / review-required case rather than collapsing review. The next step is not broad formalization; it is better tooling for identifying which tiny trust-boundary components are worth this treatment and which are not.

Finally, there is a symbiotic relationship between downstream deployment gates and upstream specification elicitation tools like *formal-specification-ide* [5]. Integrating rigorous, human-authored specifications directly into the executable oracle pipeline will radically increase the enforcement power of the triage layer.

6. Conclusion

The transition toward rigorous Secure Program Synthesis requires a corresponding shift in evaluation methodologies. Passing the visible tests is not the same as earning deployment authorization. Our findings indicate that treating visible tests as sufficient evidence for release introduces an operational vulnerability. On our 24-task suite of security-relevant Python components, hidden executable oracles systematically altered deployment decisions—catching critical failures that visible tests ignored, and reducing the secure false-accept rate from 7 to 1.

However, this study also delineates the limits of automated validation. A triage layer cannot generate code, and formal execution does not eradicate the need for human review at the boundaries of the Trusted Computing Base.

We do not claim to have fully resolved the SPS deployment problem, nor do we suggest that visible tests are without utility. Instead, we offer a pragmatic roadmap for recognizing their limits as proxies. As LLMs scale their capacity to output code, trustworthy deployment will require more than test-passing generation. It will require architecting trust-boundary deployment gates where specifications act as strict constraints: rejecting structural failures, suppressing secure false accepts, and flagging the boundaries that still require human inspection.

Appendix A: Hidden Oracle Construction and Validation

The main empirical question in this paper is whether stronger, specification-derived executable checks change deployment decisions relative to visible tests. That makes oracle construction the most important methodological surface. We therefore describe it explicitly here.

Two Hidden-Evidence Surfaces

The paper uses two kinds of hidden executable evidence, corresponding to two task surfaces.

First, on the **owned internal narrow-waist slice**, hidden checks come from the frozen day-2 harness packaged with each task. Each owned task is a versioned repo file under `data/slopbench/` and includes a visible prompt and visible test harness, alongside a day-2 prompt and day-2 test harness that encode a bounded generalization, regression, or precedence condition. These owned task files were part of the bounded v3 denominator and were frozen before confirmatory execution. For example, the visible `permission_gate` task in `030_permission_gate.yaml` checks deny-by-default behavior for explicit roles; the hidden day-2 harness adds wildcard-role expansion while preserving deny-by-default semantics. The hidden layer tightens the trust-boundary behavior that the same component is supposed to enforce.

Second, on the **secure slice**, hidden checks are generated deterministically from frozen secure-task templates. The visible surface exposes only the public summary and public constraints of each task, together with a schema-level visible test. Those public summaries are versioned in a public JSON spec

file; the frozen secure-task templates live in a secure-template module; and the hidden evaluator generation logic lives in a hidden-oracle module. The hidden oracle then evaluates a benign payload and an adversarial payload against the same function in an isolated temporary workspace. For example, the visible `safe_path_validation` surface tells the model to normalize user paths conservatively and stay within an allowed namespace; the hidden oracle then checks that an adversarial payload such as `"../secret"` is rejected rather than waved through. Each secure hidden oracle is generated from a frozen template carrying a task identifier, category, summary, and failure label, and the resulting oracle hash is recorded in the ledgers. The confirmatory split and triage policy were then held fixed rather than silently retuned after seeing results.

Separation from the Visible Harness

The visible and hidden layers are intentionally separated. Visible tests check what a model-facing benchmark would naturally expose: basic schema, core success cases, and day-1 functionality. Hidden checks ask whether the artifact still holds at the trust boundary once sharper edge cases, adversarial payloads, or day-2 regression conditions are introduced. This is the precise sense in which the paper studies visible-pass / hidden-fail behavior. The goal is not to surprise the model with arbitrary secret functionality, but to test whether a visibly plausible artifact still satisfies the stronger behavior that the bounded specification calls for.

Validation and Adjudication

The hidden oracle output is not treated as self-authenticating. Claim-bearing visible-pass / hidden-fail cases, secure false accepts, and review-boundary escalations are all backed by manual adjudication. In the packaged v3 study, deep manual adjudication covers 138 rows overall, including a secure flagship casebook and a dedicated review-boundary casebook. The role of that adjudication is to confirm that hidden failures correspond to genuine semantic or security failures, rather than harness flakiness, and that review escalations correspond to real wrapper, interface, or Trusted Computing Base ambiguities.

What This Does and Does Not Guarantee

These hidden executable checks strengthen the deployment gate, but they do not become a universal proof of correctness. They can still underconstrain some failures and overconstrain some acceptable implementations. Their purpose in this paper is narrower and more operational: to provide a stronger bounded evaluator than visible tests alone, and to make proxy-target gaps legible enough that deployment decisions can be defended.

Appendix B: Prior Work Disclosure

This submission builds on earlier exploratory work under the *SpecOracle* lineage [2], but the primary scientific object of this paper is the newer *v3* deployment-gate study developed during the SPS Hackathon. We therefore distinguish sharply between prior work used as context and the claim-bearing work under review here.

The earlier SpecOracle work should be read as background lineage. It explored whether hidden checks, stronger evaluators, and informal structural constraints could expose failure modes that visible tests miss. It also produced some reusable evaluation and orchestration substrate. That earlier work informed the direction of the project, but it is not the denominator on which the main claims of this paper rest.

The contribution under review in this submission is the hackathon-era *v3* trust-boundary study. During the hackathon, the project was re-scoped around Track 3's core question: how specifications can function as executable evaluators for autonomous code deployment. The central object of this paper — the frozen narrow-waist denominator, the trust-boundary triage policy (`auto_accept`, `auto_reject`, `escalate_to_review`), the claim-bearing ledgers, the adjudicated visible-pass / hidden-fail analysis, and the present manuscript — belongs to that newer v3 study.

Concretely, the new work for this submission includes: freezing the 24-task primary suite and corresponding expansion suite; executing the v3 triage workflow on fixed candidate pools; producing the claim-bearing synthesis, manual adjudication, and runtime provenance for the deployment-gate results; and writing the Track 3 paper around this bounded trust-boundary object. Earlier SpecOracle artifacts are retained only as historical lineage and motivation. They are not counted as new hackathon contributions, and they do not serve as the primary denominator for the empirical claims made here.

Where pre-existing repository utilities were reused, they were reused as substrate rather than as the contribution being judged. The main contribution of this submission is not the mere existence of evaluation code, but the hackathon-era freezing, reframing, execution, packaging, and analysis of the v3 deployment-gate study.

Appendix C: Limitations and Dual-Use Considerations

This appendix is narrower than the main Discussion. The paper's scientific limitations and future work are discussed above. Here we record the interpretation boundaries and misuse considerations most relevant to safe reading and reuse of the trust-boundary deployment-gate workflow.

Interpretation Boundaries

The deployment gate studied here is a bounded decision procedure for narrow-waist components, not a general deployment guarantee for arbitrary software systems. A passing triage decision should therefore not be read as "the system is safe," only as "within this bounded trust boundary, the available executable evidence did not justify rejection." Likewise, an `escalate_to_review` outcome is not a soft failure mode but an explicit acknowledgment that wrapper assumptions, interface semantics, or TCB dependencies remain outside the automated deployment surface.

The hidden executable checks in this study are stronger than the visible harness, but they are still only as strong as the specifications they operationalize. They reduce false sufficiency; they do not eliminate residual assumptions. In practice, this means the workflow can still produce false accepts, false rejects, and unresolved edge cases when the bounded specification surface and the operational deployment surface diverge. The intended use of this workflow is therefore as a stricter gate inside a deployment pipeline, not as a license to remove human review.

Dual-Use Considerations

The same analysis that helps defenders identify weak deployment proxies could also be used to study how those proxies fail. In particular, a capable actor could use similar methods to search for candidate programs that satisfy visible harnesses while violating stronger hidden requirements. That risk is real whenever visible tests are treated as the main gate for deployment.

We reduce that risk by keeping the contribution at the level of deployment-gate structure rather than exploit instruction. The paper does not present live offensive targets, reusable intrusion procedures, or operational bypass playbooks. Instead, it studies a bounded task suite and reports the resulting deployment failures, secure false accepts, candidate-support limits, and review-boundary cases at a level intended to improve defensive evaluation practice.

The practical lesson we want the community to take from this work is defensive: visible tests are often weak deployment proxies, and stronger executable evaluators should be used to block false confidence before code crosses a trust boundary. The point is to harden deployment gates, not to teach proxy gaming.

Responsible Disclosure and Ethical Considerations

This study did not target deployed third-party systems, and it did not seek to produce exploit-ready artifacts against live services. If future work applies the same methods to real external systems, the default norm should be responsible disclosure: report the issue privately to maintainers, allow a remediation window where appropriate, and prefer defensive reproductions that make the deployment-gate failure legible without unnecessarily increasing misuse risk.

The broader ethical issue is false legitimacy. Code that passes visible tests can acquire an undeserved aura of safety, especially in autonomous or semi-autonomous development settings. That is the main risk surface this paper is trying to reduce. For that reason, our triage system includes an explicit `escalate_to_review` outcome. A high-assurance workflow should not hide unresolved assumptions behind a pass signal. It should surface them clearly enough that a human can decide whether the artifact has actually earned deployment authorization.

Safety-Relevant Future Improvements

The future-work section in the main paper focuses on the scientific next steps for Secure Program Synthesis. From a safety and deployment perspective, the most important next improvements are different: stronger review-boundary tooling, clearer accounting of residual assumptions, and evaluation surfaces that make proxy-target gaps legible before code crosses a trust boundary. As stronger generators increase the volume of plausible code, these deployment-gate improvements become more important, not less.

Code and Data

- **Code and package:** `github.com/davidkimai/sps_evals`
- **Reviewer quickstart and audit path:** `runs/vericoding_research_v3/reports/reviewer_quicksta`
- **Owned internal task specs:** `data/slopbench/`
- **Visible secure task specs:** `data/vericoding_visible_secure_specs.json`
- **Frozen secure templates:** `src/specoracle/vericoding/security_checks.py`
- **Hidden oracle generation:** `src/specoracle/vericoding/hidden_oracles.py`
- **Main v3 artifacts:** `runs/vericoding_research_v3/`
- **Claim registry:** `runs/vericoding_research_v3/reports/claim_status.json`
- **Canonical ledgers:** `runs/vericoding_research_v3/ledgers/`
- **Runtime provenance:** `runs/vericoding_research_v3/inspect_logs/`

The repository README and reviewer quickstart include a minimal copy-and-paste audit path for checking the packaged tests, v3 completion contract, status report, and paper build.

Author Contributions

Jason Tang led project framing, denominator design, implementation oversight, evaluation, adjudication strategy, packaging, and paper writing.

References

[1] **Dougherty, Q.** 2026. *Secure Program Synthesis*. LessWrong sequence. Online.

[2] **Tang, J.** 2026. *SpecOracle*. GitHub repository. Online.

[3] **von Hippel, M., Henniger, S., Dougherty, Q., and Miyazono, E.** 2026. *How to Solve Secure Program Synthesis*. LessWrong. Online.

[4] **Dodds, M.** 2025. *Specifications Don't Exist*. Galois Blog. Online.

[5] **Atlas Computing.** 2026. *formal-specification-ide*. GitHub repository. Online.

[6] **OathTech.** 2026. *lean-tcb*. GitHub repository. Online.

[7] **The Lean Prover Team.** 2024. *Lean 4*. GitHub repository. Online.

[8] **Symbolic Software.** 2026. *On the Promises of "High-Assurance" Cryptography*. Online.

[9] **The Dafny Team.** 2024. *Dafny*. Online.

[10] **Bursuc, S., Ehrenborg, T., Lin, S., Astefanoaei, L., Chiosa, I. E., Kukovec, J., Singh, A., Butterley, O., Bizid, A., Dougherty, Q., Zhao, M., Tan, M., and Tegmark, M.** 2025. *A Benchmark for Vericoding: Formally Verified Program Synthesis*. arXiv:2509.22908.

[11] **Clarke, E., Grumberg, O., Jha, S., Lu, Y., and Veith, H.** 2003. *Counterexample-Guided Abstraction Refinement for Symbolic Model Checking*. Journal of the ACM.

[12] **Alur, R., Martin, M., Raghothaman, M., Stergiou, C., Tripakis, S., and Udupa, A.** 2014. *Synthesizing Finite-state Protocols from Scenarios and Requirements*. arXiv:1402.7150.

[13] **Melcer, D., Gonugondla, S., Perera, P., Qian, H., Chiang, W.-H., Wang, Y., Jain, N., Garg, P., Ma, X., and Deoras, A.** 2024. *Approximately Aligned Decoding*. arXiv:2410.01103.

[14] **Regehr, J.** 2026. *Zero-Degree-of-Freedom LLM Coding using Executable Oracles*. Online.

[15] **AI Security Institute, UK.** 2024. *Inspect AI: Framework for Large Language Model Evaluations*. Released May. Online.

LLM Usage Statement

LLM assistance was used for editing, organization, repository packaging, and consistency checks. Technical claims, commands, and metrics were checked against repository artifacts.